

THEME ARTICLE: INTELLIGENT CAD IN THE AGE OF AI

Interactive Texture Segmentation of 3D Scanned Models Leveraging Multiview Automatic Segmentation (Supplementary Material)

Koki Madono, *Waseda University / Preferred Networks, 1698555, Tokyo, Japan*

Takeo Igarashi, *Preferred Networks, 1000004, Tokyo, Japan*

Hiroharu Kato, *Preferred Networks, 1000004, Tokyo, Japan*

Taisuke Hashimoto, *Preferred Networks, 1000004, Tokyo, Japan*

Fabrice Matulic, *Preferred Networks, 1000004, Tokyo, Japan*

Tsukasa Takagi, *Preferred Networks, 1000004, Tokyo, Japan*

Keita Higuchi, *Preferred Networks, 1000004, Tokyo, Japan*

Details of Implementation

In this section, we show the details of our implementation described in the main paper.

Preprocessing

During the preprocessing phase, we calculate subsegments and subsegment distance table that records the distances between subsegments. These subsegments are utilized for interactive segmentation, enabling users to adaptively adjust the segmentation outcomes, a flexibility that is not as readily achievable with fully automatic segmentation. The precomputation of the distance table aids in accelerating the interactive segmentation process.

In 3D surface segmentation, both geometry and texture are crucial components. As confirmed in related work of main paper, 2D segmentation models typically demonstrate superior accuracy and flexibility compared to their 3D counterparts. However, just applying 2D segmentation to texture images is insufficient because it cannot capture geometric features. Consequently, in this work, we apply 2D segmentation to rendered images to improve the quality of subsegment generation P^* .

We shows an overview of boundary-based subsegment generation at Figure 1. The system first generates two boundary images, UV-extracted boundary and

3D-extracted boundary. Then the system combines the two boundaries to produce the final subsegments.

Boundary Extraction of UV Texture Map Leveraging the availability of UV texture maps, we apply a 2D segmentation model to the UV map to achieve a more comprehensive segmentation map. The UV texture is segmented through SAM-HQ [1] as follows:

$$\mathbf{S}_{UV} = \text{SAM}(T_{UV}), \quad (1)$$

where $\mathbf{S}_{UV}$ denotes a segmentation extracted from the UV texture, SAM denotes a 2D segmentation model, and T_{UV} denotes a UV texture.

The boundaries within the UV texture, denoted as $\mathbf{Bd}_{UV}$, are determined using the following equation:

$$\mathbf{Bd}_{UV} = \text{H}(\nabla \mathbf{S}_{UV}), \quad (2)$$

where $\mathbf{Bd}_{UV}$ denotes the uv-extracted boundary from the segmentation result, H denotes a heaviside step function [2], SAM denotes a 2D segmentation model, and $\nabla \mathbf{S}_{UV}$ denotes the gradient of the segmentation result derived from T_{UV} .

Boundary Extraction of Rendered Images When sampling camera positions uniformly, we often face challenges in capturing occluded regions of the 3D mesh. Moreover, mapping segmentation results onto the 3D mesh presents alignment issues due to the uncertain boundaries in 3D coordinates.

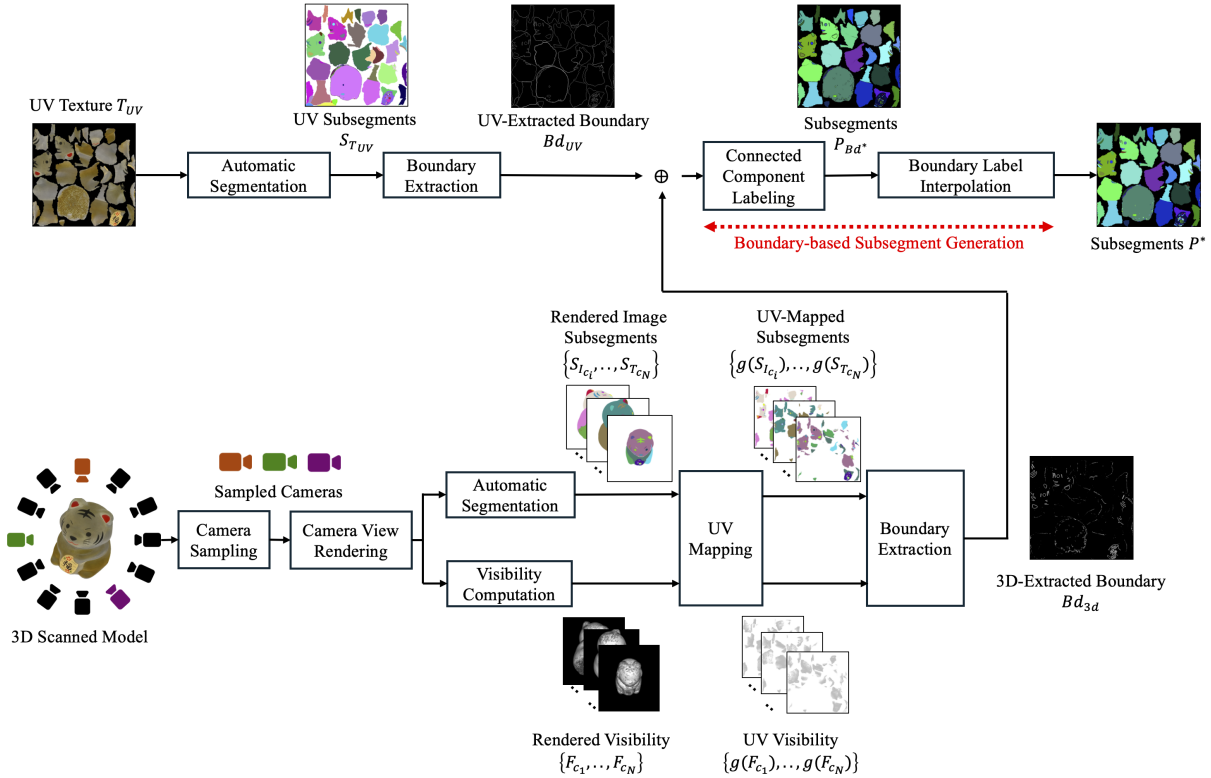


FIGURE 1. Overview of Boundary-based subsegment generation. Initially, 2D segmentation is applied to both the 3D model and the UV texture. Following this, we extract the boundaries, denoted as Bd_{3d} and Bd_{UV} , utilizing the segmentation data along with facing forward information. Next, connected component labeling (CCL) is employed to derive subsegments, indicated by P_{Bd^*} . The final step involves obtaining the refined subsegments P^* through a process of boundary interpolation.

To tackle these issues, we implement an optimization strategy for camera sampling positions. This approach specifically selects camera views that comprehensively cover the entire 3D mesh and are oriented forward. This method effectively addresses the problems associated with occluded areas and alignment uncertainties during the mapping of segmentation results onto the UV space. We specifically consider the visibility which implies how degree the mesh is facing forward to the mesh. When optimizing the camera position, we maximize the visibility of the entire mesh surface.

The degree of visibility can be calculated as follows:

$$\mathbf{F} = \text{ReLU}(\mathbf{V} \cdot \hat{\mathbf{n}}), \quad (3)$$

where $\mathbf{F}$ denotes the degree of visibility, $\mathbf{V}$ denotes the view vector, $\hat{\mathbf{n}}$ denotes the normal vector at the i -th position, and ReLU denotes the Rectified Linear Unit function, which clips negative values to zero [3].

During the optimization process, we sample N

camera positions from the entire view's camera set $\mathbf{C}$. Specifically, we employ a greedy algorithm and iteratively sample an optimal camera position that achieves the highest score.

Initially, we compute the maximum value of visibility within the iteration t as follows:

$$g(\mathbf{F}_{\mathbf{C}_t^*, (u,v)}) = \max_{i \in \mathbf{C}_t^*} g(\mathbf{F}_{\mathbf{C}_i, (u,v)}), \quad (4)$$

where $\mathbf{C}_t^*$ denotes the optimal set of cameras already sampled at the $(t + 1)$ -th iteration, g denotes the mapping function from camera coordinates to UV coordinates, $\mathbf{F}_{\mathbf{C}_i, (u,v)}$ denotes the visibility for a coordinate (u, v) observed from the i -th camera, and $\mathbf{F}_{\mathbf{C}_t^*, (u,v)}$ represents the maximum value of visibility at coordinate (u, v) within the camera set $\mathbf{C}_t^*$.

To identify the next camera for sampling, we calcu-

late the maximum visibility in UV space as follows:

$$\mathbf{C}_{t+1}^* = \arg \max_{i \in (\mathbf{C} \setminus \mathbf{C}_t^*)} \sum_{u=0}^U \sum_{v=0}^V \text{ReLU} \left(g(\mathbf{F}_{c_i, (u,v)}) - g(\mathbf{F}_{\mathbf{C}_t^*, (u,v)}) \right) \quad (5)$$

where $\mathbf{C}_{t+1}^*$ denotes the set of cameras for the $(t+1)$ -th iteration.

Once computing the optimal set of cameras $\mathbf{C}^*$, we conduct camera view rendering that obtains rendered images $\mathbf{I}_{\mathbf{C}^*}$ of each camera view.

Segmentation of the surface texture is computed using the precomputed sampled camera set $\mathbf{C}^*$. Primarily, we employ SAM-HQ [1] to obtain segmentation results for the rendered image from various camera views. Then, we compute the boundary of each segmentation as follows:

$$\mathbf{Bd}_{c_i} = \mathbf{H} \left(\nabla S_{I_{c_i}} \right), \quad (6)$$

where $\mathbf{Bd}_{c_i}$ denotes the 3d-extracted boundary generated from the i -th camera view's segmentation, I_{c_i} denotes the image captured by the i -th camera, and $S_{I_{c_i}}$ denotes the segmentation result of image I_{c_i} .

Next, we calculate the gradient of the UV texture using the following equation:

$$\nabla \mathbf{T}_{UV, (u,v)} = \zeta \left(\phi \left(\mathbf{T}_{UV, (u,v)} \right) \right) \quad (7)$$

where ϕ represents a Sobel filter applied to detect edges by calculating the gradient of the image intensity, $\mathbf{T}_{UV, (u,v)}$ denotes texture value at UV coordinates (u, v) , $\nabla \mathbf{T}_{UV, (u,v)}$ denotes gradient value at UV coordinates (u, v) , and ζ denotes a normalization function that scales the result into the $[0, 1]$ range.

Subsequently, we compute the 3D boundary using the equation below:

$$\mathbf{Bd}_{3d, (u,v)} = \mathbf{H} \left(\nabla \mathbf{T}_{(u,v)} \max_{c_i} \left(\frac{g(\mathbf{F}_{c_i, (u,v)}) \mathbf{Bd}_{c_i, (x,y)}}{g(\mathbf{F}_{\mathbf{C}^*, (u,v)})} \right) - Th_s \right) \quad (8)$$

where $\mathbf{Bd}_{3d, (u,v)}$ denotes the 3D-extracted boundary at UV coordinates (u, v) , Th_s denotes a threshold for segmentation strength. This formulation considers that boundaries emerge from both color differences and geometric features. By combining the gradients derived from the UV texture with the segmentation results obtained from multiview-rendered images, we can effectively measure the strength of these boundaries. Th_s can be set by users yet we select the optimal value from our heuristic analysis. Our selected hyperparameter considers both the quality of the subsegment and the prevention of increasing the redundant subsegments. Basically, the value we choose for the main experiment works for most cases. For boundary selection, we select the camera pixel-wise using max

operator. Since reliance on the boundary is particularly dependent on each point, pixel-wise boundary computation leads to a reasonable boundary from every camera view.

We combine the the uv-extracted boundary and 3d-extracted boundary as follows:

$$\mathbf{Bd}^* = \mathbf{Bd}_{UV} + \mathbf{Bd}_{3d}, \quad (9)$$

where $\mathbf{Bd}^*$ denotes the final boundary representation, integrating both UV and 3D boundary information to form a comprehensive boundary map. We simply adding two boundary information $\mathbf{Bd}_{UV} + \mathbf{Bd}_{3d}$. By establishing boundaries, several independent subsegments can be generated. Since the boundaries are based on segmentation information accurately segmented by SAM-HQ [1], we can considered unlikely that redundant segments are included.

Boundary-based Subsegment Generation Upon obtaining the boundary, we proceed with boundary-based subsegment generation utilizing connected-component labeling (CCL). Using fine boundary information $\mathbf{Bd}^*$, we can generate fine subsegments such as Figure 2. We describe the details in Algorithm 1. Initially, we calculate the sugsegments $\mathbf{P}_{\mathbf{Bd}^*}$ using an inverse of boundary $\mathbf{Bd}^*$ and $\mathbf{P}_{UV}$ using the UV texture binary mask $M_{T_{uv}}$ through CCL. Subsequently, the infilling regions are interpolated by iteratively employing a 3×3 kernel. This process specifically involves filling the area with the label corresponding to the nearest color. By setting a threshold Th_T , our objective is to fill areas with naturally similar colors by calculating the average color difference according to the following equation:

$$\text{Tex}((u, v), (u', v'), T_{UV}) = \frac{1}{3} \left| T_{UV, (u,v)} - T_{UV, (u', v')} \right|, \quad (10)$$

where $T_{UV, (u,v)}$ denotes the RGB color at the coordinate (u, v) on the UV texture map T_{UV} .

Subsequently, we populate the area with the label corresponding to the lowest cost function determined in Algorithm 1, which accounts for both color and geometric differences. The geometric distance is quantified using the following equation:

$$d((u, v), (u', v')) = \frac{1}{U}(u - u')^2 + \frac{1}{V}(v - v')^2, \quad (11)$$

where U and V denote the height and width of the UV texture, respectively.

Following this, the infilling area is identified by calculating the difference between the UV texture binary mask $M_{T_{uv}}$ and the mask of the filled area $M_{P_{\mathbf{Bd}^*}}$. The subsegments $\mathbf{P}_{\phi}$ are then generated through CCL. Finally, we amalgamate $\mathbf{P}_{\mathbf{Bd}^*}$ and $\mathbf{P}_{\phi}$ to form the set of generated subsegments $\mathbf{P}^*$.

Algorithm 1 Boundary-based Subsegment Generation

Require: Bd^* , ▷ Final boundary representation
 $P_{Bd^*} = CCL(1 - Bd^*)$ ▷ Apply CCL on the inverse of boundary
 $P_{UV} = CCL(M_{T_{UV}})$ ▷ Apply CCL on the UV texture binary mask
while $Tex((u, v), (u', v'), T_{UV}) < Th_T$ & $P_{Bd^*}(u, v) \in \emptyset$ **do**
 $P_{Bd^*}(u, v) = \arg \min Tex((u, v), (u', v'), T_{UV})$
end while ▷ Fill based on color difference
while $T_{(u, v)} \in P_{UV}(u', v')$ & $P_{Bd^*}(u, v) \in \emptyset$ **do**
 $P_{Bd^*}(u, v) = \arg \min Tex((u, v), (u', v'), T_{UV})d((u, v), (u', v'))$
end while ▷ Fill considering color and geometric differences
 $P_\phi = CCL(M_{T_{UV}} - M_{P_{Bd^*}})$ ▷ Fill remaining areas
 $P^* = P_{Bd^*} + P_\phi$ ▷ Combine generated subsegments

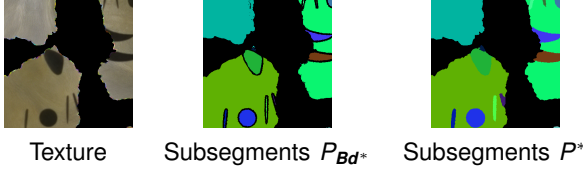


FIGURE 2. The impact of boundary-based subsegment generation: By employing boundary interpolation, the boundaries within the subsegment P_{Bd^*} were interpolated, culminating in the resulting subsegment P^* .

Figure 2 illustrates the intricacies of boundary interpolation. Initially, boundaries are not associated with specific labels, but Algorithm 1 assigns labels within the boundaries, as depicted on the right side of Figure 2.

Precomputation of Subsegment Distance Table Utilizing the subsegments P^* , we calculate the geodesic distance.

The subsegment distance G is calculated as follows:

$$G_{P_i, P_j} = \begin{cases} Tex(g(\bar{p}_{P_i}), g(\bar{p}_{P_j}), T_{UV}) d(\bar{p}_{P_i}, \bar{p}_{P_j}) & P_i \leftrightarrow P_j \\ \min_{P_k} (G_{P_i, P_j}, G_{P_i, P_k} + G_{P_k, P_j}) & \text{otherwise} \end{cases} \quad (12)$$

where G_{P_i, P_j} denotes the distance between the i -th subsegment P_i and the j -th subsegment P_j , $Tex(x, y, T_{UV})$ denotes the difference between the color of texture T_{UV} at texture coordinate x and y . $\bar{p}_{P_i}$ denotes the average 3D point within the i -th subsegment, $P_i \leftrightarrow P_j$ signifies that subsegment P_i is connected to subsegment P_j , and k indicates the index of an intermediary subsegment used for computing the minimal distance when P_i and P_j is not directly connected. This distance calculation presupposes that subsegments of similar colors and proximate locations are likely to be

classified within the same segment later during user interaction.

Run Time Processing

Our interactive segmentation system clusters subsegments into segments under the guidance of user-provided scribbles. Specifically, users assign segment IDs by drawing colored scribbles on the model, and the system propagates these segment IDs to entire subsegments based on the scribbles.

For each iteration of user interaction, the incoming scribble map in B_{c_t} where pixels under scribble have col_k which denotes k -th segmentation ID and other areas are 0 by camera c_t is integrated into the accumulated UV scribble map $B_{UV, t}$ as follows:

$$B_{UV, t, g(x, y)} = \begin{cases} col_k & B_{c_t, (x, y)} \text{ is painted} \\ B_{UV, t-1, g(x, y)} & \text{otherwise} \end{cases} \quad (13)$$

where $B_{UV, t, g(x, y)}$ denotes the scribbling value at the transformed coordinate $g(x, y)$ from camera view to UV space during the t -th iteration, $B_{c_t, (x, y)}$ denotes camera c_t 's view of scribble map for a coordinate (x, y) , and col_k denotes the k -th segmentation ID as specified by the user.

At each user iteration, the i -th subsegment's segmentation ID is updated according to the user's scribbles B_{c_t} with a following equation:

$$P_{i, t} = \begin{cases} col_k & P_{i, t} \text{ is painted by } B_{c_t} \\ P_{i, t-1} & \text{otherwise} \end{cases} \quad (14)$$

where $P_{i, t}$ denotes the segmentation ID assigned to the subsegment P_i at iteration t , and P_i denotes the i -th subsegment as computed in Algorithm 1.

Utilizing user specifications, unscribbled subsegments are automatically assigned to the closest scribbled ID using the equation:

$$k^* = \arg \min_{P_j \cap B_{UV, t} \neq \emptyset} G_{P_i, P_j} \quad (15)$$

where k^* denotes the optimal index of the segmentation ID corresponding to the nearest scribbled subsegment, and $P_j \cap \mathbf{B}_{UV,t} \neq \emptyset$ denotes whether subsegment P_j is marked as per the scribbling map $\mathbf{B}_{UV,t}$. Finally, we update the segmentation ID of subsegment P_i to denote it as k^* -th segmentation ID col_{k^*} .

REFERENCES

1. L. Ke, M. Ye, M. Danelljan, Y. Liu, Y.-W. Tai, C.-K. Tang, and F. Yu, "Segment anything in high quality," in *NeurIPS*, 2023.
2. M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*, ninth dover printing, tenth gpo printing ed. New York: Dover, 1964.
3. V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ser. ICML'10. Madison, WI, USA: Omnipress, 2010, p. 807–814.